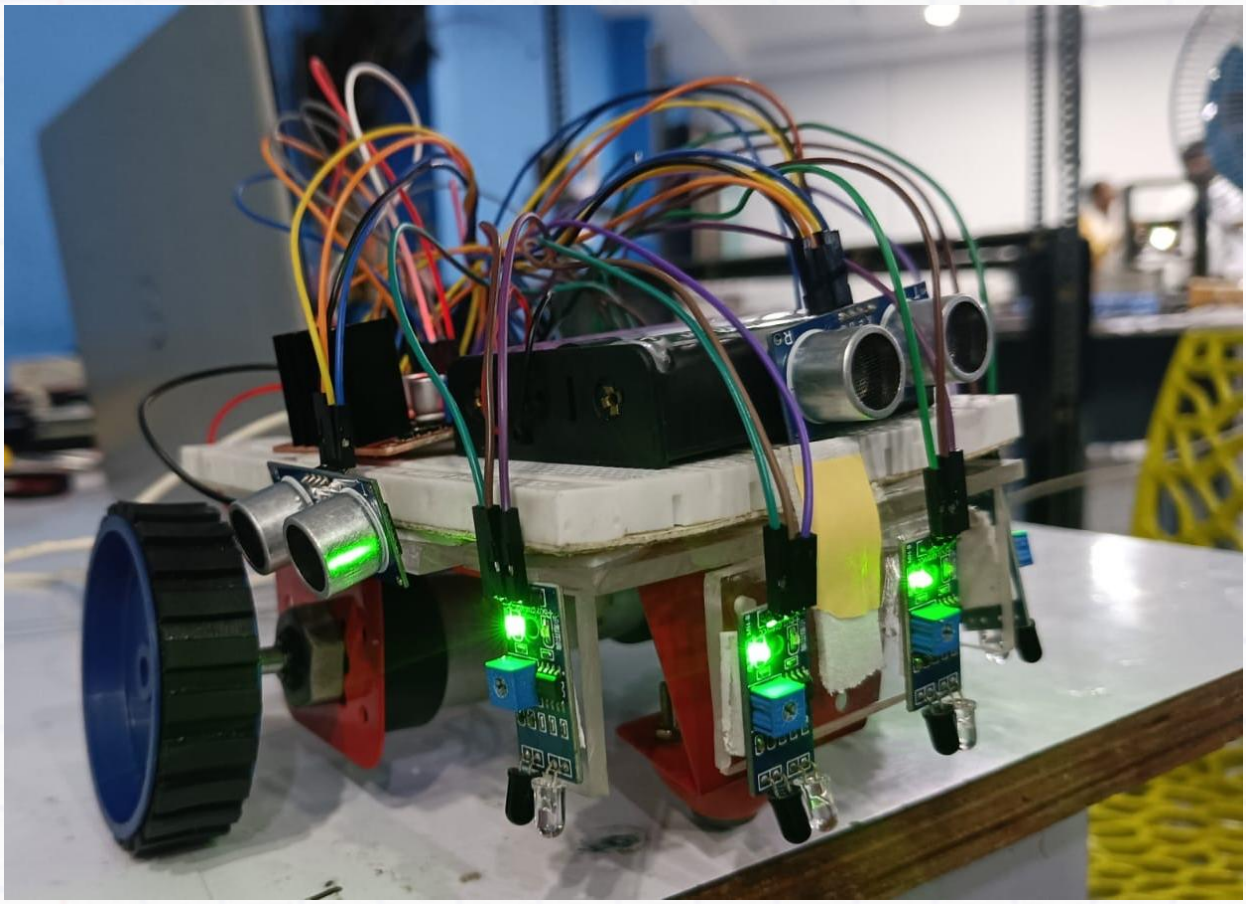
Build an Autonomous Maze Robot and a Line Tracing Robot



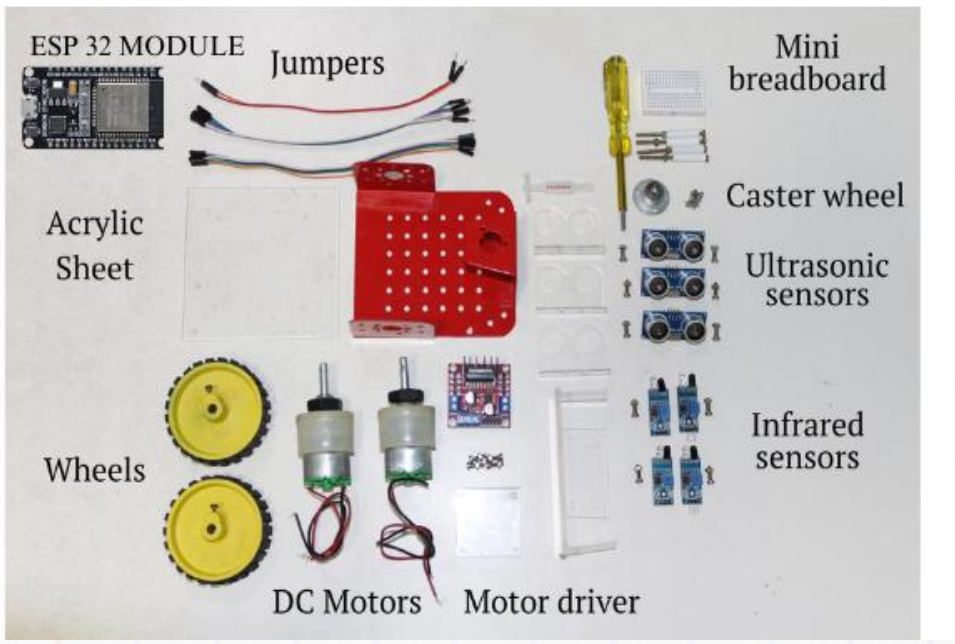
Index

Aim

a. Build an autonomous robot equipped with sensors and wheels which can autonomously navigate and solve a maze.
b. Build a line tracing robot that follows a physical path laid on ground.

Components

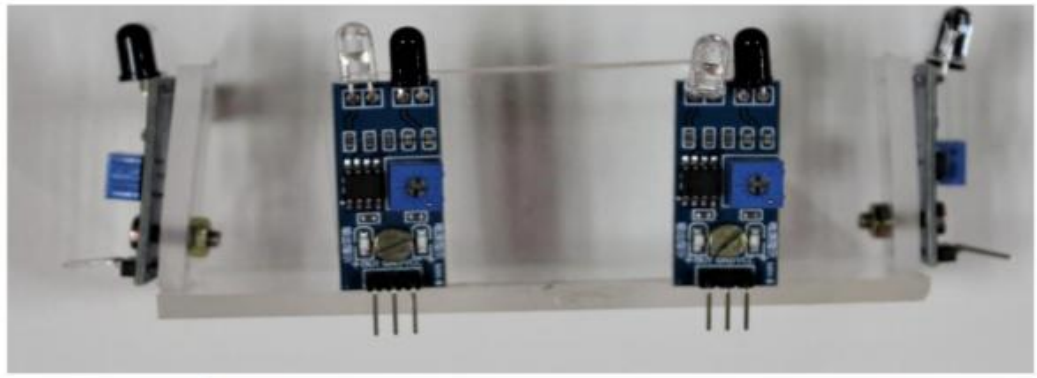
1. ESP32 C-6 Module - 1
2. IR Sensors - 4
3. Ultrasonic sensors - 3
4. L298 Motor driver - 1
5. DC Motors - 2
6. Wheels - 2
7. Caster wheel - 1
8. Metal Chassis - 1
9. Screws & nuts – As needed
10. Screwdriver - 1
11. Double sided tape - 1
12. Mini Breadboard (400 point) - 1
13. Jumper wires – As needed
15. 3.7V Li-ion battery -3
16. 1 USB cables - 1
17.3 X 18650 CELL BATTERY CASE HOLDER - 1



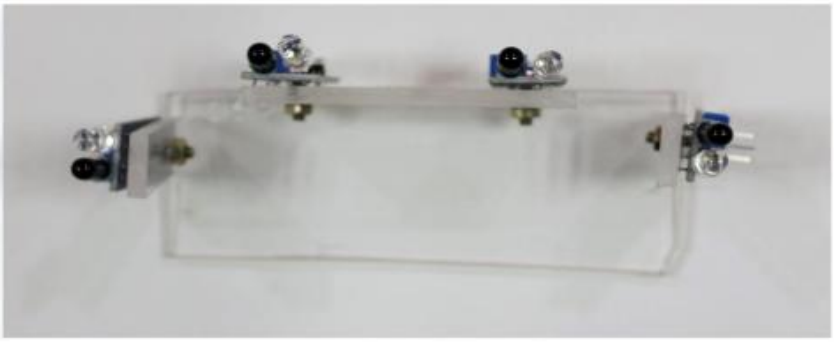
Part 1: Experimenting with the Sensors & Motors

IR Sensors

Assembly: Use some double side tape to assemble them as shown below (or directly mount the sensors on the metal chassis)



IR Sensor



IR Sensors connected on the acrylic structure

Connections:

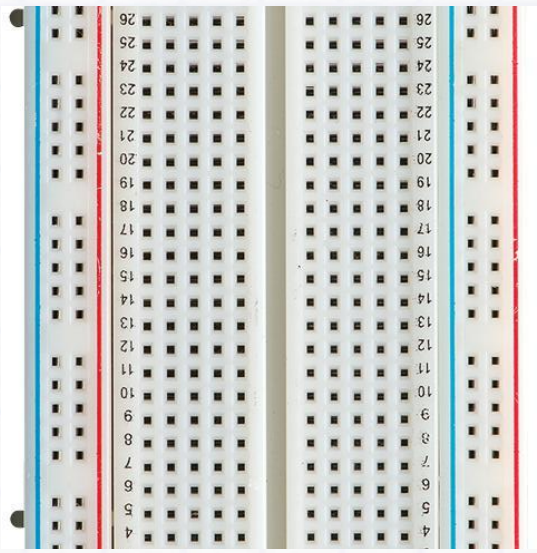
NOTE: Before starting the connections, verify using a multimeter that all the jumper wires are working. Also ensure that the connections are strong, else the setup may not work. Make sure the connections are connected at respective pins.

Take 12 female-to-male jumper wires and connect them to the pins (OUT, GND, VCC) of the 4 IR sensors. Connect the other ends as per the below connections:

Important

Sensor's Pin	Connect to...			
	Left sensor	Left_centre sensor	Right_centre sensor	Right sensor
OUT	2	10	11	3
GND	Breadboard Row R2	Breadboard Row R2	Breadboard Row R2	Breadboard Row R2
VCC	Breadboard Row R1	Breadboard Row R1	Breadboard Row R1	Breadboard Row R1

Take 2 male-to-male jumper wires and connect them as follows:



This documentation provides detailed information on the robot control system code, which includes checking IR sensors, ultrasonic sensors, and controlling DC motors for various movements. The pin connections for each component and the code structure are explained.

Pin Connections

IR Sensors:

- **IR_SENSOR_1:** GPIO 10
- **IR_SENSOR_2:** GPIO 11
- **IR_SENSOR_3:** GPIO 2
- **IR_SENSOR_4:** GPIO 3

Ultrasonic Sensors:

Ultrasonic Sensor 1:
- **TRIG_PIN_1:** GPIO 13
- **ECHO_PIN_1:** GPIO 12

Ultrasonic Sensor 2:
- **TRIG_PIN_2:** GPIO 4
- **ECHO_PIN_2:** GPIO 5

Ultrasonic Sensor 3:
- **TRIG_PIN_3:** GPIO 6
- **ECHO_PIN_3:** GPIO 7

DC Motors:

- **Motor A:**
  - **enA:** GPIO 22
  - **in1:** GPIO 19 (RX2)
  - **in2:** GPIO 18 (TX2)
- **Motor B:**
  - **enB:** GPIO 23
  - **in3:** GPIO 20
  - **in4:** GPIO 21

Breadboard Connections

****IR Sensors:****

1. Connect IR_SENSOR_1 (GPIO 10) to a digital input pin on the breadboard.
2. Connect IR_SENSOR_2 (GPIO 11) to a digital input pin on the breadboard.
3. Connect IR_SENSOR_3 (GPIO 2) to a digital input pin on the breadboard.
4. Connect IR_SENSOR_4 (GPIO 3) to a digital input pin on the breadboard.
5. Ensure each sensor is connected to VCC (3.3V or 5V depending on the sensor specification) and GND.

Ultrasonic Sensors:

1. Connect TRIG_PIN_1 (GPIO 13) to the trigger pin of the first ultrasonic sensor.
2. Connect ECHO_PIN_1 (GPIO 12) to the echo pin of the first ultrasonic sensor.
3. Connect TRIG_PIN_2 (GPIO 4) to the trigger pin of the second ultrasonic sensor.
4. Connect ECHO_PIN_2 (GPIO 5) to the echo pin of the second ultrasonic sensor.
5. Connect TRIG_PIN_3 (GPIO 6) to the trigger pin of the third ultrasonic sensor.
6. Connect ECHO_PIN_3 (GPIO 7) to the echo pin of the third ultrasonic sensor.
7. Ensure each sensor is connected to VCC (5V) and GND.

DC Motors:

1. Connect enA (GPIO 22) to the enable pin of Motor A on the motor driver.
2. Connect in1 (GPIO 19) and in2 (GPIO 18) to the input pins of Motor A on the motor driver.
3. Connect enB (GPIO 23) to the enable pin of Motor B on the motor driver.
4. Connect in3 (GPIO 20) and in4 (GPIO 21) to the input pins of Motor B on the motor driver.
5. Connect the motor driver to an appropriate power supply and GND.

Code Structure

IR Sensor Check:

The IR sensor check code initialises the IR sensors as inputs and continuously reads their values, printing them to the Serial Monitor.

Code Explanation

Pin Definitions:

- **const int IR_SENSOR_1 = 10;** defines the GPIO pin number for the first IR sensor.
- **const int IR_SENSOR_2 = 11;** defines the GPIO pin number for the second IR sensor.
- **const int IR_SENSOR_3 = 2;** defines the GPIO pin number for the third IR sensor.
- **const int IR_SENSOR_4 = 3**; defines the GPIO pin number for the fourth IR sensor.

Setup Function:

- **Serial.begin(115200);** initializes the serial communication at 115200 baud rate, which is used for debugging.
- **pinMode(IR_SENSOR_1, INPUT);** sets the pin connected to the first IR sensor as an input.
- **pinMode(IR_SENSOR_2, INPUT);** sets the pin connected to the second IR sensor as an input.
- **pinMode(IR_SENSOR_3, INPUT);** sets the pin connected to the third IR sensor as an input.
- **pinMode(IR_SENSOR_4, INPUT);** sets the pin connected to the fourth IR sensor as an input.

Loop Function:

- **int sensorValue1 = digitalRead(IR_SENSOR_1);** reads the digital value from the first IR sensor (HIGH or LOW).
- **int sensorValue2 = digitalRead(IR_SENSOR_2);** reads the digital value from the second IR sensor (HIGH or LOW).
- **int sensorValue3 = digitalRead(IR_SENSOR_3);** reads the digital value from the third IR sensor (HIGH or LOW).
- **int sensorValue4 = digitalRead(IR_SENSOR_4);** reads the digital value from the fourth IR sensor (HIGH or LOW).

Serial Printing:

- **Serial.print("Sensor 1: ")**; prints the label "Sensor 1: " to the Serial Monitor.
- **Serial.print(sensorValue1);** prints the value of the first IR sensor.
- **Serial.print("\t");** prints a tab character for formatting.
- **Serial.print("Sensor 2: ");** prints the label "Sensor 2: " to the Serial Monitor.
- **Serial.print(sensorValue2)**; prints the value of the second IR sensor.
- **Serial.print("\t");** prints a tab character for formatting.
- **Serial.print("Sensor 3: ");** prints the label "Sensor 3: " to the Serial Monitor.
- **Serial.print(sensorValue3);** prints the value of the third IR sensor.
- **Serial.print("\t");** prints a tab character for formatting.
- **Serial.print("Sensor 4: ");** prints the label "Sensor 4: " to the Serial Monitor.
- **Serial.println(sensorValue4);** prints the value of the fourth IR sensor and moves to the next line.

Delay:

- **delay(500);** introduces a delay of 500 milliseconds before the next iteration of the loop.

Create a checkIRsensor.ino file and copy the below code and paste it on the created file.

```
// Define IR sensor pins
const int IR_SENSOR1 = 10;
const int IR_SENSOR2 = 11;
const int IR_SENSOR3 = 2;
const int IR_SENSOR4 = 3;

void setup() {
  Serial.begin(115200);

  pinMode(IR_SENSOR1, INPUT);
  pinMode(IR_SENSOR2, INPUT);
  pinMode(IR_SENSOR3, INPUT);
  pinMode(IR_SENSOR4, INPUT);
}

void loop() {
  int irState1 = digitalRead(IR_SENSOR1);
  int irState2 = digitalRead(IR_SENSOR2);
  int irState3 = digitalRead(IR_SENSOR3);
  int irState4 = digitalRead(IR_SENSOR4);

  Serial.print("IR1: "); Serial.print(irState1);
  Serial.print(" | IR2: "); Serial.print(irState2);
  Serial.print(" | IR3: "); Serial.print(irState3);
  Serial.print(" | IR4: "); Serial.println(irState4);

  delay(200); // Slow down readings for visibility
}
```

Ultrasonic Sensor Check:

The ultrasonic sensor check code initialises the sensors, measures distances, and prints the values to the Serial Monitor.

Code Explanation

Pin Definitions:

- **const int TRIG_PIN_1 = 13;** defines the GPIO pin number for the trig pin of the first ultrasonic sensor.
- **const int ECHO_PIN_1 = 12;** defines the GPIO pin number for the echo pin of the first ultrasonic sensor.
- Similar definitions are provided for the second and third ultrasonic sensors.

Maximum Distance Definition:

- **const long MAX_DISTANCE = 10;** sets the maximum distance (in centimeters) for the sensors to be considered "in range".

Distance Measurement Function:

- **long measureDistance(int trigPin, int echoPin)** is a function that takes the trig and echo pin numbers as arguments and returns the measured distance.
- Inside this function:
    1. The trig pin is cleared by setting it to LOW for 2 microseconds.
    2. The trig pin is set to HIGH for 10 microseconds to trigger the ultrasonic pulse.
    3. The echo pin is read to get the duration of the returned pulse in microseconds.
    4. The duration is converted to distance using the formula duration * 0.034 / 2.

Setup Function:

- **Serial.begin(115200);** initializes serial communication at a baud rate of 115200 for debugging purposes.
- pinMode is used to set the trig pins as outputs and the echo pins as inputs for all three sensors.

Loop Function:

- measureDistance is called for each sensor to get their respective distances.
- The distances are printed to the Serial Monitor.
- If the distance is greater than the maximum distance, "Out of range" is printed.
- There is a 1-second delay before the loop repeats to take the next measurement.
- This code reads the distances from three ultrasonic sensors and prints their values to the Serial Monitor every second.

Create a checkUltraSonicsensor.ino file and copy the below code and paste it on the created file.

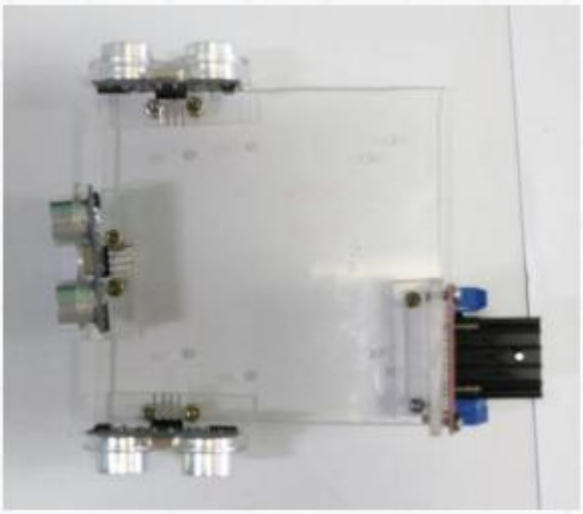
```
// Define Ultrasonic sensor pins
const int trigPin1 = 6;  // Right Sensor
const int echoPin1 = 7;
const int trigPin2 = 4;  // Left Sensor
const int echoPin2 = 5;
const int trigPin3 = 13;  // Front Sensor
const int echoPin3 = 12;
// Function to measure distance
long getDistance(int trigPin, int echoPin) {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  return pulseIn(echoPin, HIGH) * 0.034 / 2;
}
void setup() {
  Serial.begin(115200);

  pinMode(trigPin1, OUTPUT);
  pinMode(echoPin1, INPUT);
  pinMode(trigPin2, OUTPUT);
```

```
  pinMode(echoPin2, INPUT);
  pinMode(trigPin3, OUTPUT);
  pinMode(echoPin3, INPUT);
}
void loop() {
  long distanceRight = getDistance(trigPin1, echoPin1);
  long distanceLeft = getDistance(trigPin2, echoPin2);
  long distanceFront = getDistance(trigPin3, echoPin3);

  Serial.print("Right: "); Serial.print(distanceRight); Serial.print(" cm, ");
  Serial.print("Left: "); Serial.print(distanceLeft); Serial.print(" cm, ");
  Serial.print("Front: "); Serial.print(distanceFront); Serial.println(" cm");

  delay(500); // Update every half second
}
```



Tasks

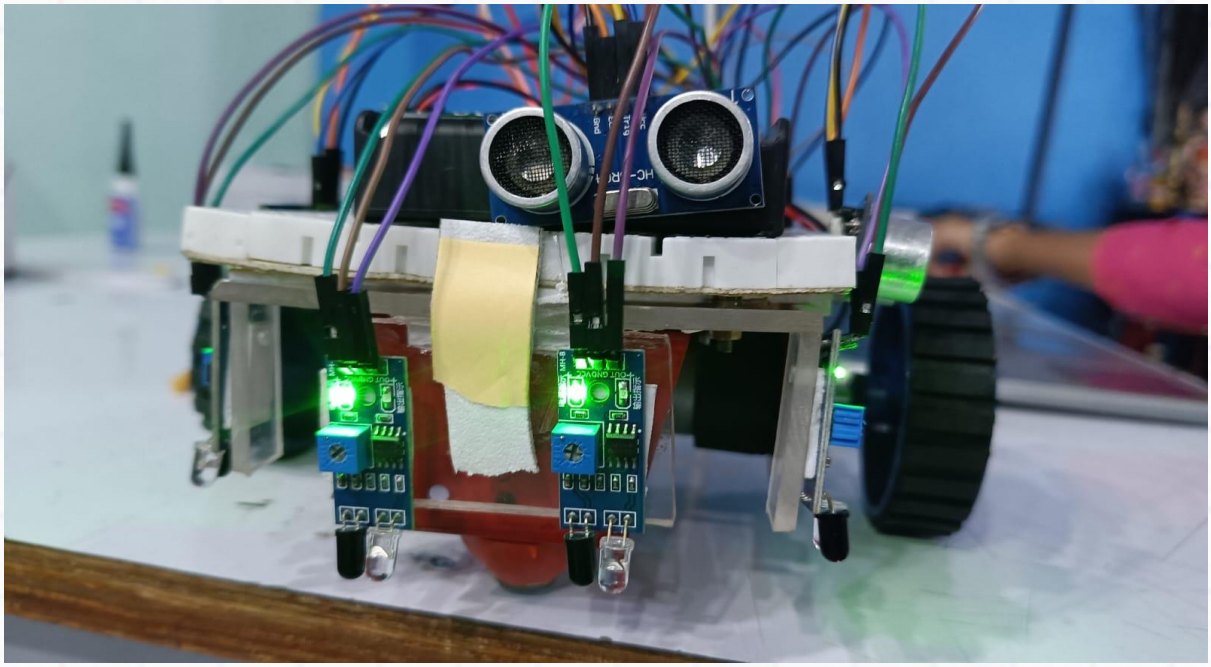
Place a 30cm ruler on the floor perpendicular to a wall so that the 0cm mark touches the wall.

Keep any one Ultrasonic sensor at the 10cm mark and verify if the reading in Live Expressions is also approximately 10c.

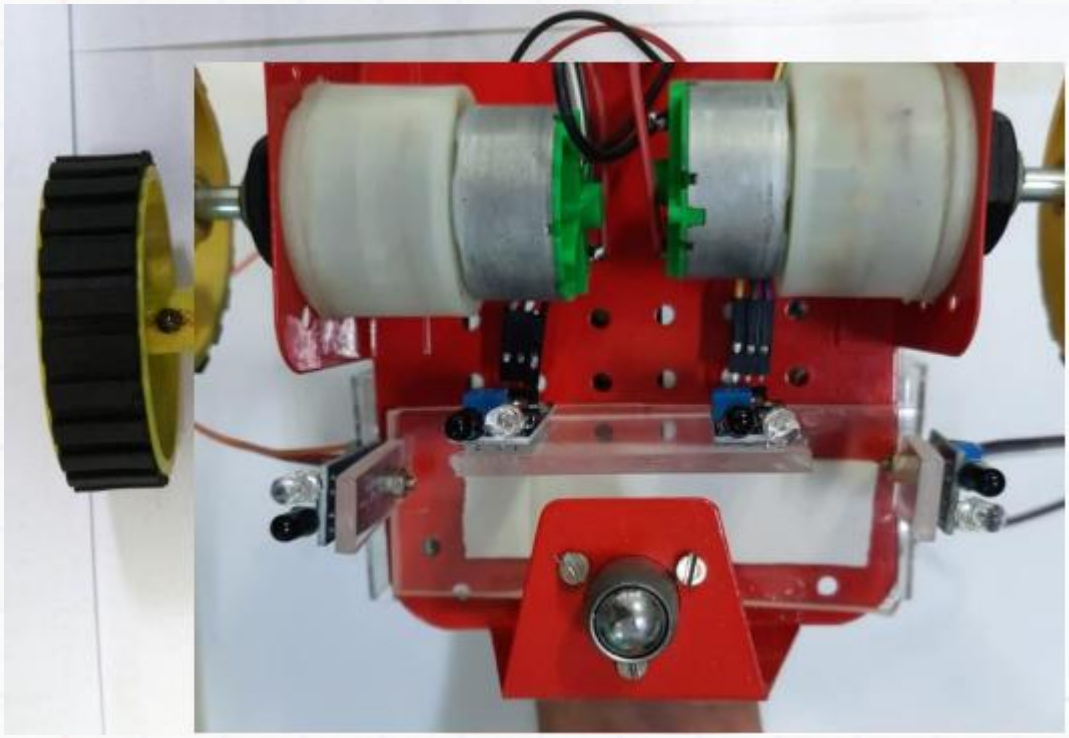
Take 3 readings for each sensor (say at 20cm, 30cm, 17cm) to verify that they are working properly.

If the distance readings for the sensors are not accurate, go to the top of the app.c file. In the Calibration Values section, you will find a variable called ultrasonic calibration set to the value 53. Adjust this value until the readings are accurate

(Note: You must test with different distances to ensure accuracy). 2) Keep one of the Ultrasonic sensors at the 20cm mark. Now slowly keep moving the sensor backwards. Note at what distance the readings lose their accuracy. This is the sensor's maximum range of functioning.



Motor Control:



The motor control code provides functions to control the motion of the robot, including moving forward, backward, and turning left or right.

Code Explanation

Pin Definitions:

- **int enA = 22;** defines the PWM pin for Motor A.
- **int in1 = 19;** and **int in2 = 18;** define the control pins for Motor A.
- Similar definitions are provided for Motor B.

PWM Configuration:

- **int pwmChannelA = 1;** and **int pwmChannelB = 1**; define the PWM channels for Motors A and B (Note: using the same channel for both motors will not work; each motor should have a unique channel).
- **int pwmFrequency = 5000;** sets the PWM frequency to 5000 Hz.
- **int pwmResolution = 8;** sets the PWM resolution to 8 bits.

Setup Function:

- **pinMode** sets the motor control pins as outputs.
- **ledcSetup** configures the PWM channels with the specified frequency and resolution.
- **ledcAttachPin** attaches the PWM channels to the GPIO pins.
- **Serial.begin(9600);** initializes serial communication at 9600 baud.
- **digitalWrite** ensures that the motors are initially turned off.

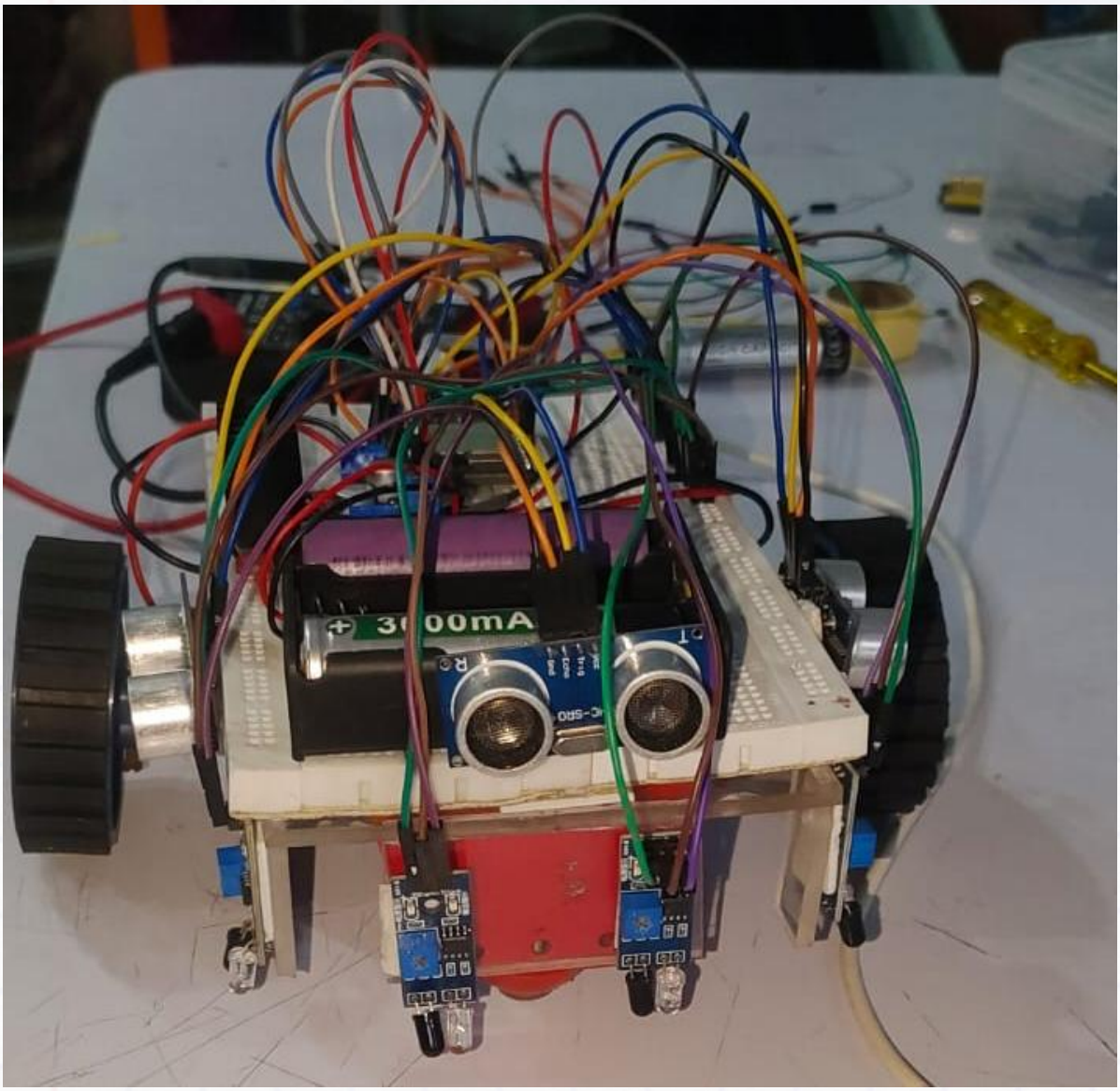
Loop Function:

The loop function calls the **forwardMotion, backwardMotion, leftMotion,** and **rightMotion** functions in sequence with delays in between.

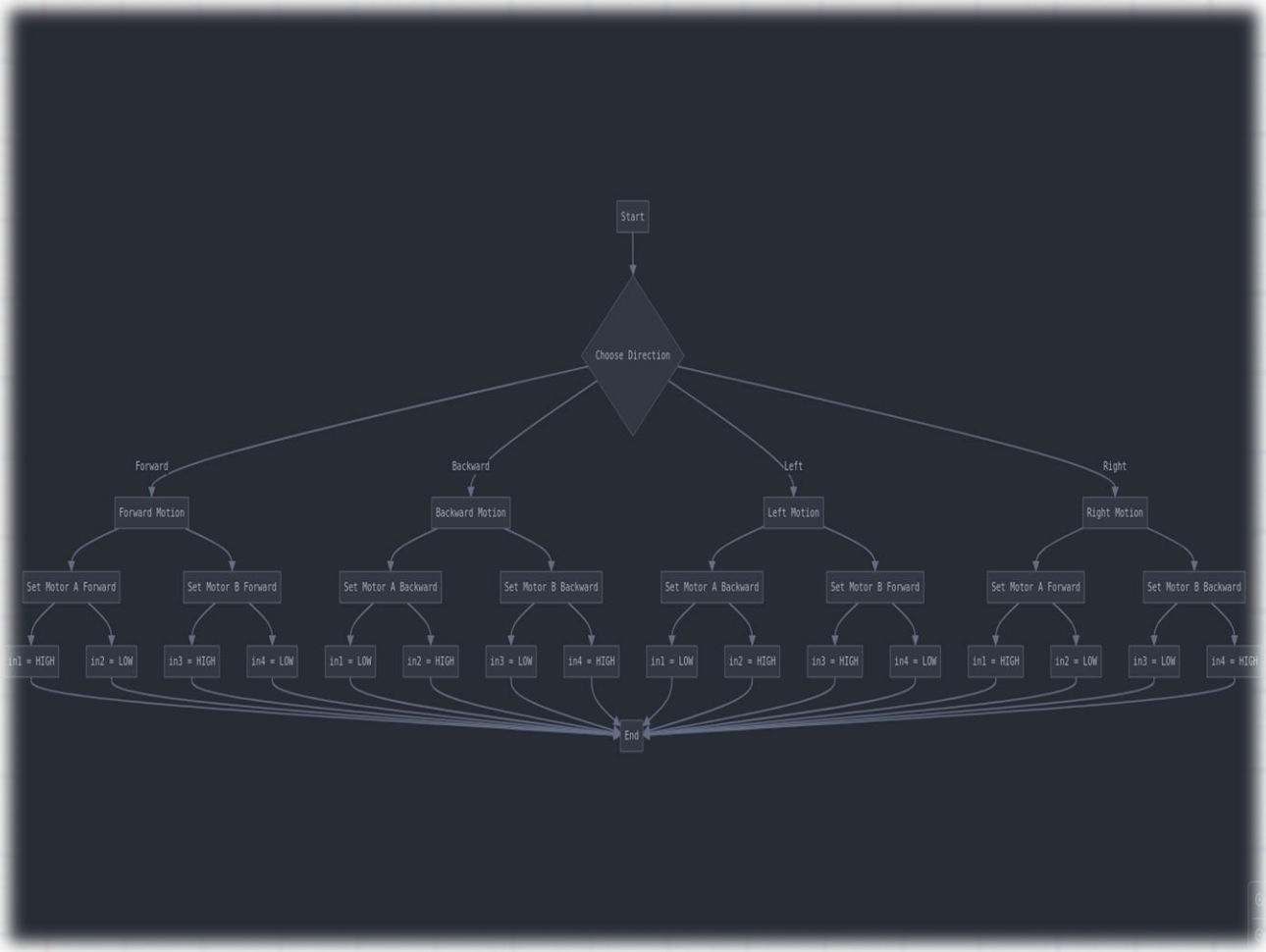
Motor Control Functions:

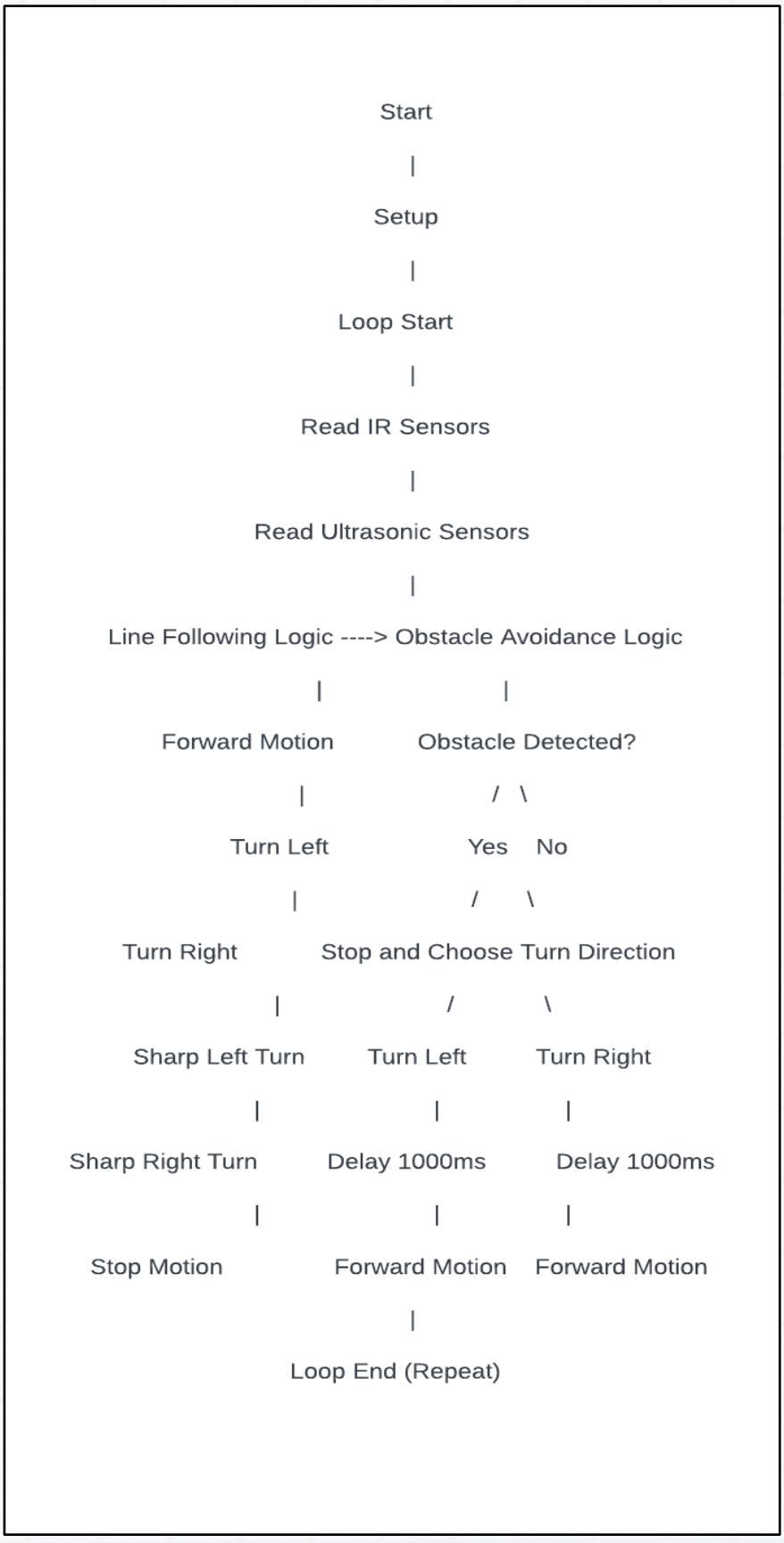
forwardMotion, backwardMotion, leftMotion, and rightMotion control the direction and speed of the motors by setting the appropriate control pins and PWM values. Create a motorCheck.ino file and copy the below code and paste it on your file.

```
// Define motor control pins
int motor1pin1 = 19;
int motor1pin2 = 18;
int motor2pin1 = 20;
int motor2pin2 = 21;
// Speed control pins
int ENA = 22; // PWM for motor 1
int ENB = 23; // PWM for motor 2
void setup() {
  Serial.begin(115200);
  pinMode(motor1pin1, OUTPUT);
  pinMode(motor1pin2, OUTPUT);
  pinMode(motor2pin1, OUTPUT);
  pinMode(motor2pin2, OUTPUT);
  pinMode(ENA, OUTPUT);
  pinMode(ENB, OUTPUT);
}
void loop() {
  Serial.println("Moving Forward");
  digitalWrite(motor1pin1, HIGH); digitalWrite(motor1pin2, LOW);
  digitalWrite(motor2pin1, HIGH); digitalWrite(motor2pin2, LOW);
  analogWrite(ENA, 150); analogWrite(ENB, 150);
  delay(2000);
  Serial.println("Turning Left");
  digitalWrite(motor1pin1, LOW); digitalWrite(motor1pin2, HIGH);
  digitalWrite(motor2pin1, HIGH); digitalWrite(motor2pin2, LOW);
  delay(2000);
  Serial.println("Turning Right");
  digitalWrite(motor1pin1, HIGH); digitalWrite(motor1pin2, LOW);
  digitalWrite(motor2pin1, LOW); digitalWrite(motor2pin2, HIGH);
  delay(2000);
  Serial.println("Stopping");
  digitalWrite(motor1pin1, LOW); digitalWrite(motor1pin2, LOW);
  digitalWrite(motor2pin1, LOW); digitalWrite(motor2pin2, LOW);
  delay(2000);
}
```


+ 3 00mA

FLOW DIAGRAM




Start
Setup
Loop Start
Read IR Sensors
Read Ultrasonic Sensors
Line Following Logic ----> Obstacle Avoidance Logic
Forward Motion
Obstacle Detected?
Turn Left
Yes
No
Turn Right
Stop and Choose Turn Direction
Sharp Left Turn
Turn Left
Turn Right
Sharp Right Turn
Delay 1000ms
Delay 1000ms
Stop Motion
Forward Motion
Forward Motion
Loop End (Repeat)

After Checking the sensor use below code for maze solver:

Part 2: Build a Maze Solving Robot

Below Tabular column represents the condition to be check on line following logic. Make sure edit the below given code accordingly

Sensor 1	**Sensor 2**	**Sensor 3**	**Sensor 4**	**Action**
LOW	HIGH	HIGH	LOW	Move forward
HIGH	LOW	-	-	Turn left
-	-	LOW	HIGH	Turn right
LOW	LOW	HIGH	HIGH	Sharp left turn
HIGH	HIGH	LOW	LOW	Sharp right turn
-	-	-	-	Stop

Below Tabular column represents the actions of the motor A and B. Use it as a reference and write it the motor control function accordingly.

Function	in1	in2	in3	in4	pwmChannelA	pwmChannelB
forwardMotion	HIGH	LOW	HIGH	LOW	200	200
backwardMotion	LOW	HIGH	LOW	HIGH	200	200
leftMotion	LOW	HIGH	HIGH	LOW	200	200
rightMotion	HIGH	LOW	LOW	HIGH	200	200
stopMotion	LOW	LOW	LOW	LOW	0	0
sharpLeftMotion	LOW	HIGH	HIGH	LOW	255	255
sharpRightMotion	HIGH	LOW	LOW	HIGH	255	255

Create a new sketch in Arduino IDE named ***mazeSolver.ino, Copy*** the below code and paste it on created file, edit with given instructions.

```
// --- Motor Control Pins ---
int motor1pin1 = 19;
int motor1pin2 = 18;
int motor2pin1 = 20;
int motor2pin2 = 21;
int ENA = 22;  // PWM Motor 1
int ENB = 23;  // PWM Motor 2
// --- Ultrasonic Sensor Pins ---
const int trigPinRight = 6;
const int echoPinRight = 7;
const int trigPinLeft = 4;
const int echoPinLeft = 5;
const int trigPinFront = 13;
const int echoPinFront = 12;
// --- IR Sensor Pins ---
const int IR_SENSOR1 = 10;
const int IR_SENSOR2 = 11;
const int IR_SENSOR3 = 2;
const int IR_SENSOR4 = 3;
// --- Setup ---
void setup() {
  Serial.begin(115200);
  // Motor Pins
  pinMode(motor1pin1, OUTPUT);
  pinMode(motor1pin2, OUTPUT);
  pinMode(motor2pin1, OUTPUT);
  pinMode(motor2pin2, OUTPUT);
  pinMode(ENA, OUTPUT);
  pinMode(ENB, OUTPUT);
  // Ultrasonic Sensor Pins
  pinMode(trigPinRight, OUTPUT);
  pinMode(echoPinRight, INPUT);
  pinMode(trigPinLeft, OUTPUT);
  pinMode(echoPinLeft, INPUT);
  pinMode(trigPinFront, OUTPUT);
  pinMode(echoPinFront, INPUT);
  // IR Sensor Pins
  pinMode(IR_SENSOR1, INPUT);
  pinMode(IR_SENSOR2, INPUT);
  pinMode(IR_SENSOR3, INPUT);
```

```
  pinMode(IR_SENSOR4, INPUT);
}
// --- Motor Functions ---
void MoveForward() {
  digitalWrite(motor1pin1, HIGH); digitalWrite(motor1pin2, LOW);
  digitalWrite(motor2pin1, HIGH); digitalWrite(motor2pin2, LOW);
  analogWrite(ENA, 150); analogWrite(ENB, 150);
}
void TurnLeft() {
  digitalWrite(motor1pin1, LOW); digitalWrite(motor1pin2, HIGH);
  digitalWrite(motor2pin1, HIGH); digitalWrite(motor2pin2, LOW);
  analogWrite(ENA, 150); analogWrite(ENB, 150);
}
void TurnRight() {
  digitalWrite(motor1pin1, HIGH); digitalWrite(motor1pin2, LOW);
  digitalWrite(motor2pin1, LOW); digitalWrite(motor2pin2, HIGH);
  analogWrite(ENA, 150); analogWrite(ENB, 150);
}
void StopMotor() {
  digitalWrite(motor1pin1, LOW); digitalWrite(motor1pin2, LOW);
  digitalWrite(motor2pin1, LOW); digitalWrite(motor2pin2, LOW);
}
// --- Ultrasonic Distance Function ---
long getDistance(int trigPin, int echoPin) {
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  return pulseIn(echoPin, HIGH) * 0.034 / 2;
}
// --- Main Loop ---
void loop() {
  // Read IR Sensors
  int ir1 = digitalRead(IR_SENSOR1);
  int ir2 = digitalRead(IR_SENSOR2);
  int ir3 = digitalRead(IR_SENSOR3);
  int ir4 = digitalRead(IR_SENSOR4);
  // Read Distances
  long frontDist = getDistance(trigPinFront, echoPinFront);
  long leftDist  = getDistance(trigPinLeft, echoPinLeft);
  long rightDist = getDistance(trigPinRight, echoPinRight);
  // Debug Info
```

```
  Serial.print("IR1: "); Serial.print(ir1);
  Serial.print(" | IR2: "); Serial.print(ir2);
  Serial.print(" | IR3: "); Serial.print(ir3);
  Serial.print(" | IR4: "); Serial.print(ir4);
  Serial.print(" || Front: "); Serial.print(frontDist);
  Serial.print(" cm | Left: "); Serial.print(leftDist);
  Serial.print(" cm | Right: "); Serial.println(rightDist);
  // --- Obstacle Avoidance Logic ---
  if (frontDist < 15) {
    StopMotor();
    Serial.println("Obstacle Ahead");
    if (rightDist > 15) {
      Serial.println("Turning Right");
      TurnRight(); delay(500);
    } else if (leftDist > 15) {
      Serial.println("Turning Left");
      TurnLeft(); delay(500);
    } else {
      Serial.println("No Path. Stopping.");
      StopMotor();
    }
  }
  // --- IR Sensor Line/Direction Logic ---
  else if (ir1 == 1 && ir2 == 1) {
    Serial.println("IR Forward Signal");
    MoveForward();
  } else if (ir3 == 1) {
    Serial.println("IR Left Signal");
    TurnLeft(); delay(300); MoveForward();
  } else if (ir4 == 1) {
    Serial.println("IR Right Signal");
    TurnRight(); delay(300); MoveForward();
  } else {
    Serial.println("No IR Signal - Stopping");
    StopMotor();
  }
  delay(100);
}
```

Here's an explanation of the code:

Constants and Variables

1. **IR Sensors:**

- `const int IR_SENSOR_1 = 10;`
- `const int IR_SENSOR_2 = 11;`
- `const int IR_SENSOR_3 = 2;`
- `const int IR_SENSOR_4 = 3;`

- These constants define the GPIO pins connected to the four IR sensors.

2. **Ultrasonic Sensors:**

- `const int TRIG_PIN_1 = 13;`
- `const int ECHO_PIN_1 = 12;`
- `const int TRIG_PIN_2 = 4;`
- `const int ECHO_PIN_2 = 5;`
- `const int TRIG_PIN_3 = 6;`
- `const int ECHO_PIN_3 = 7;`

- These constants define the GPIO pins connected to the trigger and echo pins of three ultrasonic sensors.

3. **Motor Connections:**

- `int enA = 22;`
- `int in1 = 19;`
- `int in2 = 18;`
- `int enB = 23;`
- `int in3 = 20;`
- `int in4 = 21;`

- These integers define the GPIO pins connected to the motor driver for controlling two motors.

4. **PWM Configuration:**

- `int pwmChannelA = 1;`
- `int pwmChannelB = 2;`
- `int pwmFrequency = 5000;`
- `int pwmResolution = 8;`

- These integers configure the PWM channels, frequency, and resolution for controlling motor speed.

5. **Maximum Distance:**
   - `const long MAX_DISTANCE = 30;`
   - This constant defines the maximum distance (in centimeters) for obstacle detection using ultrasonic sensors.

`setup()` Function

The `setup()` function runs once when the microcontroller starts.

1. **Serial Communication:**
   - `Serial.begin(115200);`
   - Initializes serial communication at a baud rate of 115200 for debugging.

2. **IR Sensor Pins:**
   - Sets the IR sensor pins as inputs using `pinMode()`.

3. **Ultrasonic Sensor Pins:**
   - Sets the ultrasonic sensor trigger pins as outputs and echo pins as inputs using `pinMode()`.

4. **Motor Control Pins:**
   - Sets the motor control pins as outputs using `pinMode()`.

5. **PWM Configuration:**
   - `ledcSetup()` configures the PWM channels.
   - `ledcAttachPin()` attaches the PWM channels to the motor enable pins.

6. **Initial Motor State:**
   - Calls `stopMotion()` to ensure motors are stopped at the start.

`loop()` Function

The `loop()` function runs continuously after the `setup()` function.

1. **Read IR Sensors:**
   - Reads the values from the IR sensors using `digitalRead()`.

2. **Read Ultrasonic Sensors:**

- Calls `measureDistance()` to get the distance from each ultrasonic sensor.

3. **Line Following Logic:**
- Depending on the IR sensor values, calls functions to move forward, turn left/right, or make sharp turns.

4. **Obstacle Avoidance Logic:**
- If an obstacle is detected within `MAX_DISTANCE` in front, the robot stops and then turns left or right based on the distances measured by the side ultrasonic sensors.

5. **Delay:**
- A short delay before the next loop iteration to prevent excessive processing.

`measureDistance()` Function

The `measureDistance()` function measures the distance using an ultrasonic sensor.

1. **Clear `trigPin`:**
- Sets the `trigPin` to LOW for 2 microseconds to ensure a clean signal.

2. **Trigger the Sensor:**
- Sets the `trigPin` to HIGH for 10 microseconds to send the ultrasonic pulse.

3. **Read Echo:**
- Reads the duration of the echo pulse on the `echoPin` using `pulseIn()`.

4. **Calculate Distance:**
- Converts the duration to distance in centimeters and returns it.

Motor Control Functions

These functions control the motor directions and speeds.

1. **`forwardMotion()`:**
- Moves the robot forward by setting motor A and B to move forward with a PWM value of 200.

2. **`backwardMotion()`:
- Moves the robot backward by setting motor A and B to move backward with a PWM value of 200.

3. **`leftMotion()`:
- Turns the robot left by setting motor A backward and motor B forward with a PWM value of 200.

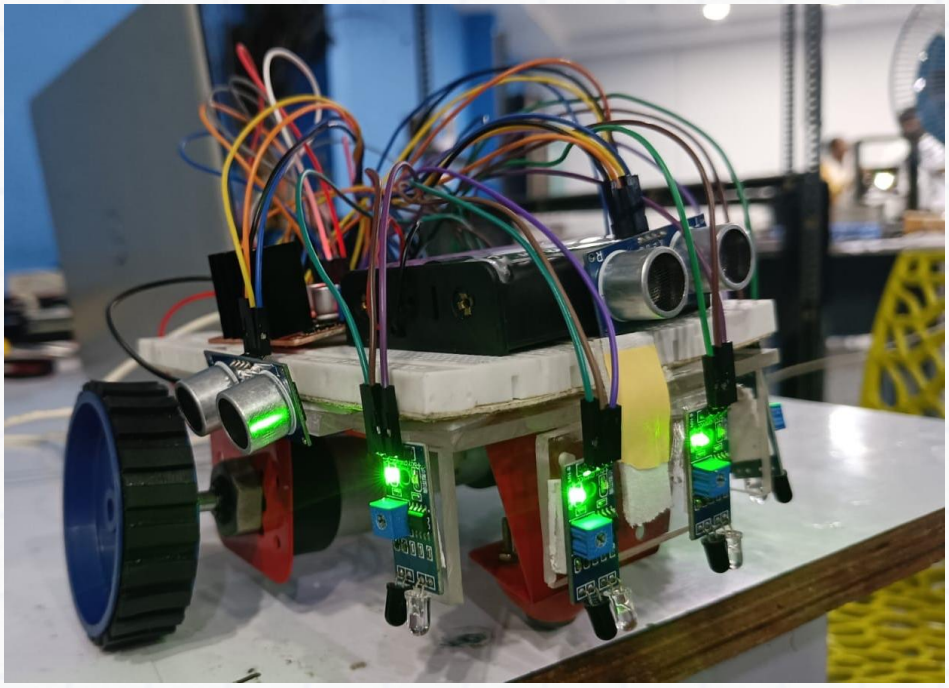
4. **`rightMotion()`:
- Turns the robot right by setting motor A forward and motor B backward with a PWM value of 200.

5. **`stopMotion()`:
- Stops the robot by setting all motor control pins to LOW and PWM values to 0.

6. **`sharpLeftMotion()`:
- Makes a sharp left turn by setting motor A backward and motor B forward with a maximum PWM value of 255.

7. **`sharpRightMotion()`:
- Makes a sharp right turn by setting motor A forward and motor B backward with a maximum PWM value of 255.

This code integrates line following and obstacle avoidance using IR and ultrasonic sensors, respectively, and controls the robot's movements accordingly.



Workshop Task

1) Build a Line Following Robot. On a white surface, draw a path using black tape. Using only the centre 2 IR sensors to detect the black line, the Micro-mouse should be able to follow it. Note: The black tape width must be less than the gap between the centre 2 IR sensors. Don't create abrupt turns in the black line, else the robot may not detect it.

Post Workshop Tasks (Club Activity)

1) Build the fastest and most efficient Maze Solver! Write programs for implementing various popular maze solving algorithms such as:
a) Left-hand following b) Right-hand following Explore more such algorithms on the internet. Eg. Wikipedia Note down how many attempts it takes to solve the maze for each algorithm. Which one was the best?

2) Build a Maze Learning Robot! Devise a program such that the Micro-mouse is made to repeatedly attempt the maze until it is solved. In the process of traversing the maze, the robot should learn the maze and map it in its memory. Therefore, once fully mapped, the Micro-mouse should be able to solve the maze without making any mistake in a single attempt.